

# 第 1 章：Transformer、Tokenization 与自回归预训练

## 本章符号说明

| 符号/缩写              | English                  | 中文             | 含义                          |
|--------------------|--------------------------|----------------|-----------------------------|
| Transformer        | Transformer Architecture | Transformer 架构 | 基于 self-attention 的序列建模架构。  |
| BPE                | Byte Pair Encoding       | 字节对编码          | 常见 subword tokenization 方法。 |
| MHA                | Multi-Head Attention     | 多头注意力          | 在多个子空间并行计算 attention 的模块。   |
| $X$                | Input Embedding Matrix   | 输入嵌入矩阵         | 一段 token embedding 组成的矩阵。   |
| $Q / K / V$        | Query / Key / Value      | 查询 / 键 / 值     | attention 中由 $X$ 线性投影得到的表示。 |
| $d_k$              | Key Dimension            | key 向量维度       | attention 缩放项中的维度。          |
| $P(w_t / w_{< t})$ | Next-token Probability   | 下一个 token 概率   | 自回归语言模型的训练目标。               |
| CE                 | Cross Entropy            | 交叉熵            | 预测 token 分布与真实 token 的损失。   |

## 本章目标

- 理解 tokenization 为什么重要。
- 掌握 self-attention 的矩阵公式。
- 区分 encoder-only、decoder-only、encoder-decoder。
- 理解 next-token prediction 为什么能学到通用能力。
- 能解释 pretraining loss 和 inference decoding。

## Tokenization

LLM 不能直接处理字符串，通常先把文本切成 token，再映射成 embedding。

常见 tokenization 方法：

- word-level：按词切分，词表大，未登录词问题明显。
- char-level：按字符切分，词表小，但序列很长。

- subword：折中方案，BPE、WordPiece、SentencePiece 都属于这一类。

面试表达：

Tokenization 决定了模型看到的最小语义单位，也影响上下文长度、压缩率、多语言表现和代码能力。中文、代码、数学表达式通常对 tokenizer 更敏感。

## Self-Attention

输入 embedding 矩阵为：

$$X \in \mathbb{R}^{n \times d}$$

通过线性投影得到：

$$Q = XW_Q, K = XW_K, V = XW_V$$

scaled dot-product attention：

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})V$$

直觉：

- $Q$  表示当前位置想找什么信息。
- $K$  表示每个位置能被什么信息匹配。
- $V$  表示真正被聚合的内容。
- 除以  $\sqrt{d_k}$  是为了避免点积随维度变大而过大，使 softmax 过于尖锐。

## Multi-Head Attention

multi-head attention 让模型在多个子空间里计算 attention：

$$head_i = Attention(XW_i^Q, XW_i^K, XW_i^V)$$

然后拼接：

$$MHA(X) = Concat(head_1, \dots, head_h)W^O$$

面试直觉：

不同 head 可以关注不同关系，例如局部语法、长程依赖、实体指代、代码括号匹配。它不是保证每个 head 都有清晰可解释含义，但增加了并行建模不同关系的能力。

## Decoder-only LLM

主流生成式 LLM 多用 decoder-only Transformer。训练目标是自回归 next-token prediction：

$$\max_{\theta} \sum_{t=1}^T \log P_{\theta}(w_t / w_{<t})$$

对应最小化 cross entropy：

$$L(\theta) = - \sum_{t=1}^T \log P_{\theta}(w_t / w_{<t})$$

为了防止模型看到未来 token，会使用 causal mask：

$$mask_{ij}=0 \text{ if } j \leq i$$

## 为什么 next-token prediction 有用

next-token prediction 看起来只是预测下一个词，但要做好它，模型需要隐式学习：

- 语法和语义。
- 世界知识。
- 长程依赖。
- 代码和数学模式。
- 指令和对话格式。

但它也有天然限制：

- 目标是拟合语料分布，不等于遵循用户意图。
- 训练目标不直接惩罚幻觉。
- 模型可能学到偏见、噪声和过期知识。

这就是为什么后续还需要 SFT、RLHF、DPO 和 RAG。

## Inference Decoding

推理时，模型会根据当前上下文预测下一个 token，然后把生成 token 拼回上下文继续生成。

常见 decoding 策略：

| 方法          | English              | 中文       | 直觉                     |
|-------------|----------------------|----------|------------------------|
| Greedy      | Greedy Decoding      | 贪心解码     | 每步选概率最高 token，稳定但容易死板。 |
| Beam Search | Beam Search          | 束搜索      | 保留多个候选，适合翻译，不一定适合开放生成。 |
| Temperature | Temperature Sampling | 温度采样     | 调整分布尖锐程度。              |
| Top-k       | Top-k Sampling       | Top-k 采样 | 只从最高的 $k$ 个 token 采样。  |

| 方法    | English          | 中文  | 直觉                         |
|-------|------------------|-----|----------------------------|
| Top-p | Nucleus Sampling | 核采样 | 从累计概率达到 $p$ 的 token 集合中采样。 |

## 本章面试高频问题

### 1. Transformer 相比 RNN 的核心优势是什么？

Transformer 用 attention 直接建模任意 token 间关系，并且训练时可以并行处理序列位置；RNN 顺序递推，长程依赖和并行效率都更弱。

### 2. 为什么 attention 要除以 $\sqrt{d_k}$ ？

点积的方差会随维度变大而增大，如果不缩放，softmax 会过于尖锐，梯度变小，训练不稳定。除以  $\sqrt{d_k}$  可以让 attention logits 的尺度更稳定。

### 3. decoder-only 为什么适合生成？

decoder-only 使用 causal mask，只能看当前位置之前的 token，天然匹配从左到右生成。GPT 系列、LLaMA 系列、Qwen 系列都属于这一路线。

### 4. next-token prediction 为什么能产生 few-shot 能力？

大规模语料里存在大量任务描述、示例、问答、代码和推理模式。模型在预训练中学习这些分布后，可以在 prompt 中根据少量示例适配新任务。

## 本章 References

- [Attention Is All You Need](#)
- [Language Models are Few-Shot Learners](#)
- [FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

[章节索引](#)

[← 上一章](#) [下一章 →](#)